Date: .....

Expt. No.: ..... Page No.: .....

Chapter → 11
Inheritance & More On Loops

Inheritance: is a way of creating of a new class from an existing class.

Syntax:

```
class Employee:          # Base class
    #code
class Programmer(Employee):    # Derived or child class
    #code
```

We can also use the method and attributes of 'Employee' in 'Programmer' object.

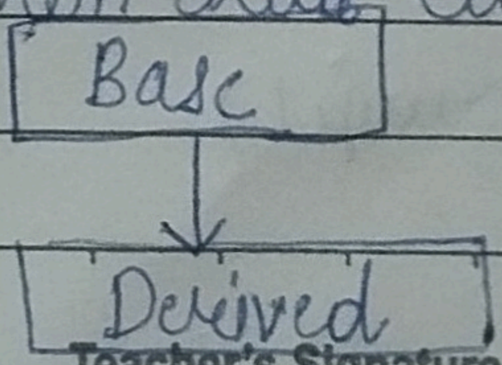
Also, we can overwrite or add new attributes and methods in 'Programmer' class.

• Types of Inheritance

1. Single Inheritance
2. Multiple Inheritance
3. Multilevel Inheritance

• Single Inheritance

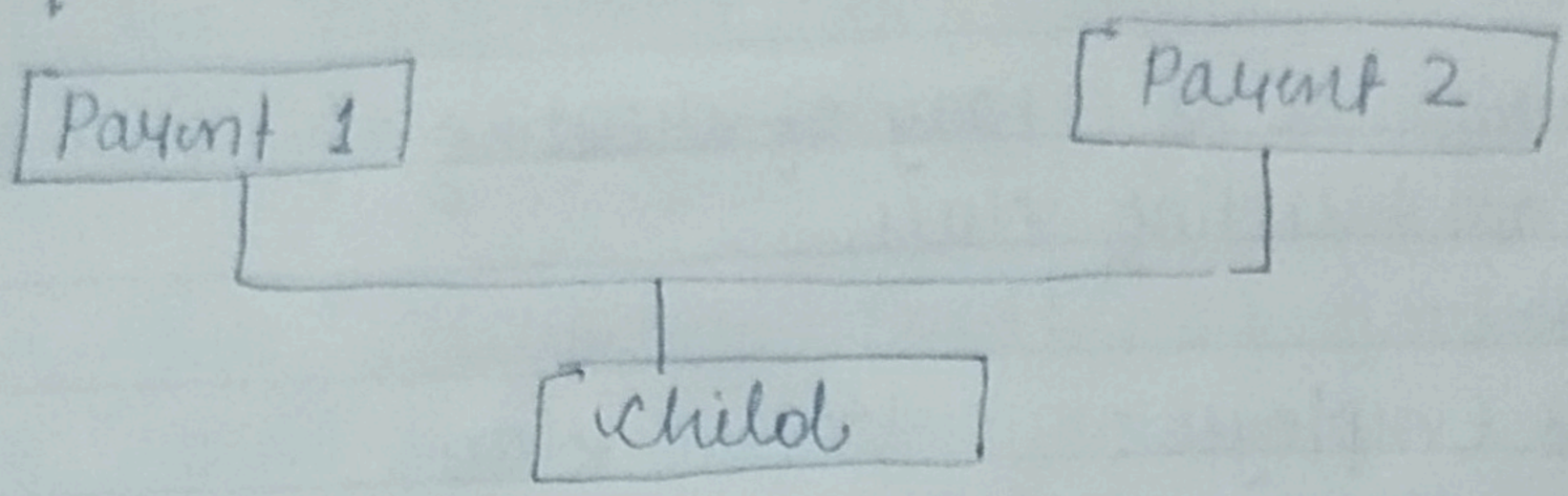
Single Inheritance occurs when child class inherits only a Single Parent class.



Teacher's Signature _____

• Multiple Inheritance

Multiple Inheritance occurs when the child class inherits from more than one parent class.



• Examples of Single & Multiple Inheritance

//Code of Single Inheritance

```
class Employee:
        company = "ITC", salary = "50K"
        def show(self):
            print(f"The name of the Company is
                {self.company} and the salary is
                    {self.salary}")

class Programmer:
        company = "ITC Infotech", language = "python"
        def showLanguage(self):
            print(f"The name is {self.company} and he is
                good with {self.language} language")

a = Employee()"ITC", "50K")
b = Programmer("ITC Infotech' Python")
print(a.company, b.company)
b.show()
b.showLanguage()
```

Date: .....

Expt. No.: .....

Page No.: .....

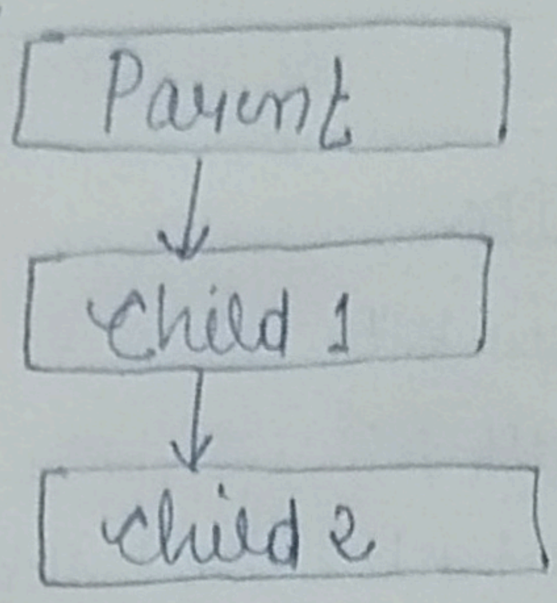
// Code of Multiple Inheritance

```
class Employee:
    company = "ITC"
    Salary = "50k"
    def show(self):
        print(f"The name of the company is {self.company} and the salary is {self.salary}")
class Coder:
    language = "python"
    def printLanguages(self):
        print(f"out of all the languages here is your favourite language: {self.language}")
class Programmer(Employee, Coder):
    company = "ITC Infotech"
    def showLanguage(self):
        print(f"The name is {self.company} and he is good with {self.language} language")
a = Employee()
b = Coder()
c = Programmer()
c.show()
c.printLanguages()
c.showLanguage()
```

Teacher's Signature _____

• Multilevel Inheritance

:hen a child class becomes a parent for another child class.



• Basic Syntax of Multilevel Inheritance

```
class Employee:
        a = 1
class Programmer (Employee):
        b = 2
class Manager (Programmer):
        c = 3
O = Employee ( )
print (O.a)

O = Programmer ( )
print(O.a, O.b)

O = Manager ( )
print (O.a, O.b, O.c)
```

Date: .....

Expt. No.: .....

Page No.: .....

• Super() Method.

Super() method is used to access the methods of a super class in the derived class.

super().__init__()

#__init__() calls constructor of the base class

• Class Method.

A class method is a method which is bound to the class and not the object of the class.

@classmethod decorator is used to create a class method.

Syntax:

@classmethod

def(cls, P1, P2):

• Property Decorators

Consider the following class:

@Property

def name(self):

return f"{self.fname} {self.lname}"

@name.setter

def name(self, value):

self.fname = value.split(" ")[0]

self.lname = value.split(" ")[1]

e = Employee()

e.a = "Malvip Singer"

Print(e.fname, e.lname)

e.show()

Teacher's Signature _____

Operators Overloading in Python

Operators in Python can be overloaded using dunder methods.
These methods are called, when a given operator is used on the objects.
Operators in Python can be overloaded using the following methods.

```
p1+p2   # p1.__add__(p2)
p1-p2   # p1.__sub__(p2)
p1*p2   # p1.__mul__(p2)
p1/p2   # p1.__truediv__(p2)
p1//p2  # p1.__floordiv__(p2)
```

Other dunder/magic methods in Python:

```
__str__()  # used to set what gets displayed upon calling str(obj).
__len__()  # used to set what gets displayed upon calling .__len__() or len(obj).
```

Date: .....

Expt. No.: ..... Page No.: .....

Chapter - 11
Practice - pdf

1) Create a class (2-D vector) and use it to create another class representing a 3-D vector.

2) Create a class 'Pets' from a class 'Animals' and further create a class 'Dog' from 'Pets'. Add a method 'bark' to class 'Dog'.

3) Create a class 'Employee' and add salary and increment properties to it.
Write a method 'salaryAfterIncrement' method with a @Property decorator with a setter which changes the value of increment based on salary.

4) Write a class 'complex' to represent complex numbers along with overloaded operators '+' and '*' which adds and multiply them.

5) Write a class vector representing a vector of n dimensions. Overload the + and * operator which calculates the sum and dot (.) product of them.

6) Write - Str - () method to print the vector as follows:
$7i + 8j + 10k$
Assume vector of dimension 3 for this problems.

Teacher's Signature _____